

Lecture 6: Data Acquisition - Scraping Dynamic Websites

James Sears*

AFRE 891/991 FS 26

Michigan State University

*Parts of these slides are adapted from [“Data Science for Economists”](#) by Grant McDermott.

Table of Contents

1. Prologue
2. Scraping Dynamic Websites

Prologue

Packages for Today

Today we'll pick up with scraping dynamic, client-side websites.

For this we'll need the following packages:

Static:

```
pacman::p_load(lubridate, rvest, stringr, tiydverse, tidyr)
```

and for dynamic scraping:

```
pacman::p_load(chromote, selenider)
```

Scraping Dynamic Websites

Dynamic Websites

Dynamic client-side websites are those are rendered dynamically and require **interaction** in order to obtain desired information.

While **rvest** can help us retrieve urls and element contents, it can't interact with **dynamically-generated or interactive** websites.

To do that, we're going to take advantage of **browser automation** methods.

Browser Automation

The main idea behind browser automation is this:

If a website requires pointing/clicking/typing to reveal the contents we want, why not have software "drive" a browser for us?

Old Approach: RSelenium

- R bindings for the Selenium webdriver
- Requires some additional software (i.e. Docker)
- Additional steps to get it working with Mac OS X

New Approach: selenider

- Uses **chromote** or Selenium automation tools to drive a Chrome browser
- Syntax more closely matches tidyverse
- Can't do some more complex things yet (i.e. click-and-hold, click a point)

rvest vs. Selenium

As websites are increasingly moving toward server-side with site protection protocols, dynamic scraping methods are becoming increasingly necessary.

As a result, **rvest** has been expanding into methods for dynamic web scraping.

However, these methods are still experimental/being developed, whereas **selenium** is currently much more powerful - so we'll be focusing on **selenium** methods.

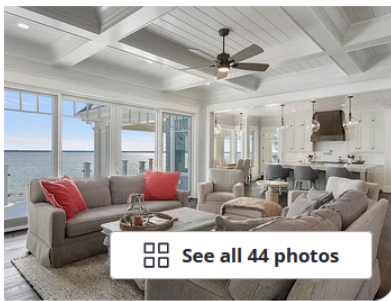
Application: Traverse City Properties

Let's work through an example of an interactive workflow for **property characteristics and tax info in Traverse City**.

Suppose, hypothetically, that you just came into a hefty inheritance. You've decided the best course of action is to blow it all on a nice new place up north in Traverse City.

Application: Traverse City Properties

After a little time scrolling real estate websites, you find a nice candidate:



\$8,875,000
16768 Wrightwood Terrace Dr, Traverse City, MI 49686

Est.: \$52,729/mo [Get pre-qualified](#)

5 beds
6 baths
8,440 sqft

[Request a tour](#)
as early as today at 12:30 pm

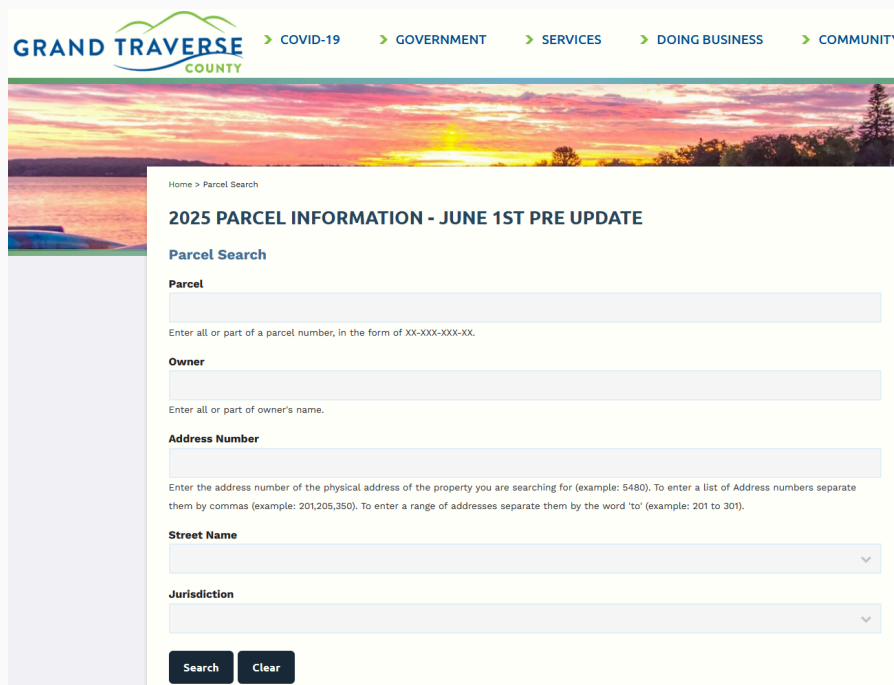
[Contact agent](#)

[See all 44 photos](#)

Application: Traverse City Properties

While you can find a decent amount of property info on the real estate site's page, you want to make sure that this would be a "responsible" purchase by verifying the tax assessed value and school district.

So, you navigate to the [Grand Traverse County Parcel Info Website](#) to see if you can find the info.



The screenshot shows the Grand Traverse County website's parcel search interface. At the top, the logo for Grand Traverse County is displayed alongside navigation links for COVID-19, GOVERNMENT, SERVICES, DOING BUSINESS, and COMMUNITY. Below the navigation bar is a scenic image of a sunset over water. The main content area is titled "2025 PARCEL INFORMATION - JUNE 1ST PRE UPDATE" and "Parcel Search". It contains several input fields: "Parcel" (with a placeholder "Enter all or part of a parcel number, in the form of XX-XXX-XXX-XX"), "Owner" (with a placeholder "Enter all or part of owner's name."), "Address Number" (with a placeholder "Enter the address number of the physical address of the property you are searching for (example: 5480). To enter a list of Address numbers separate them by commas (example: 201,205,350). To enter a range of addresses separate them by the word 'to' (example: 201 to 301)."), "Street Name" (a dropdown menu), and "Jurisdiction" (a dropdown menu). At the bottom of the form are "Search" and "Clear" buttons.

Application: Traverse City Properties

After searching for the address, you find a little bit of information, including the **parcel number (APN)**.

Home > Parcel Search

2025 PARCEL INFORMATION - JUNE 1ST PRE UPDATE

Parcel Search

 SEARCH

1 item found.

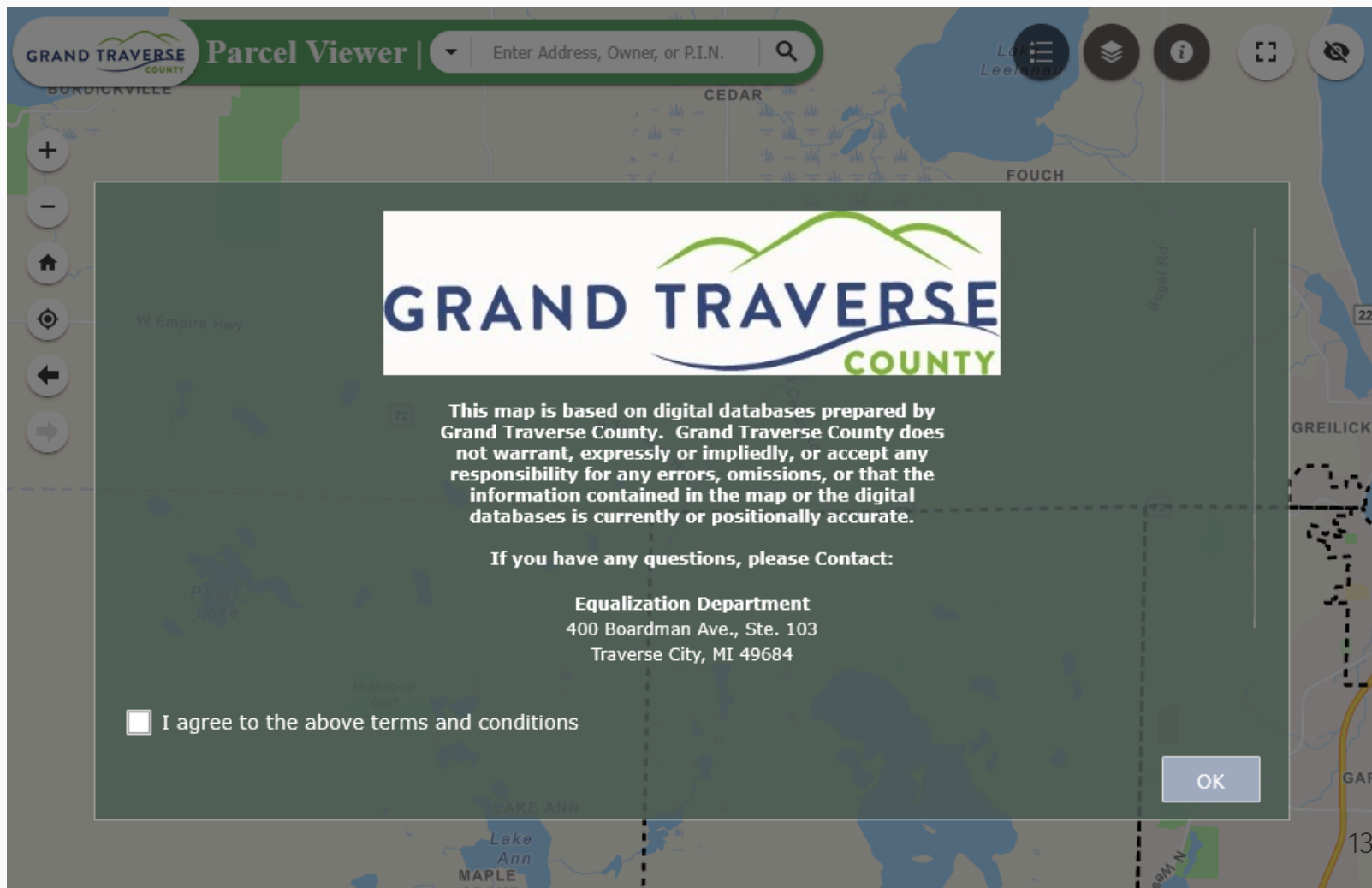
You searched for items with Address Number = 16768, Street Name = WRIGHTWOOD TERRACE DR.

Parcel ▲	Owner ▲	Address ▲	City ▲
11-111-007-25	SNIDER MANAGEMENT TRUST	16768 WRIGHTWOOD TERRACE DR	TRAVERSE CITY

You recall that Grand Traverse City has a [Parcel Map Viewer Website](#) that has a search by APN feature and contains a lot of information - including school district and property tax values!

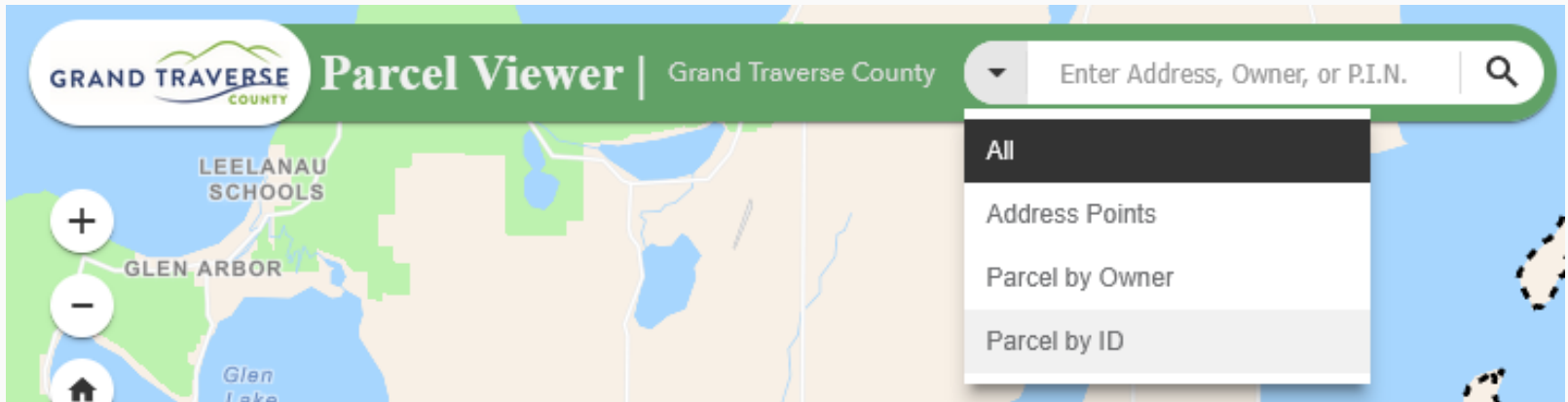
Application: Traverse City Properties

So, you navigate over to the [Parcel Map Viewer Website](#):



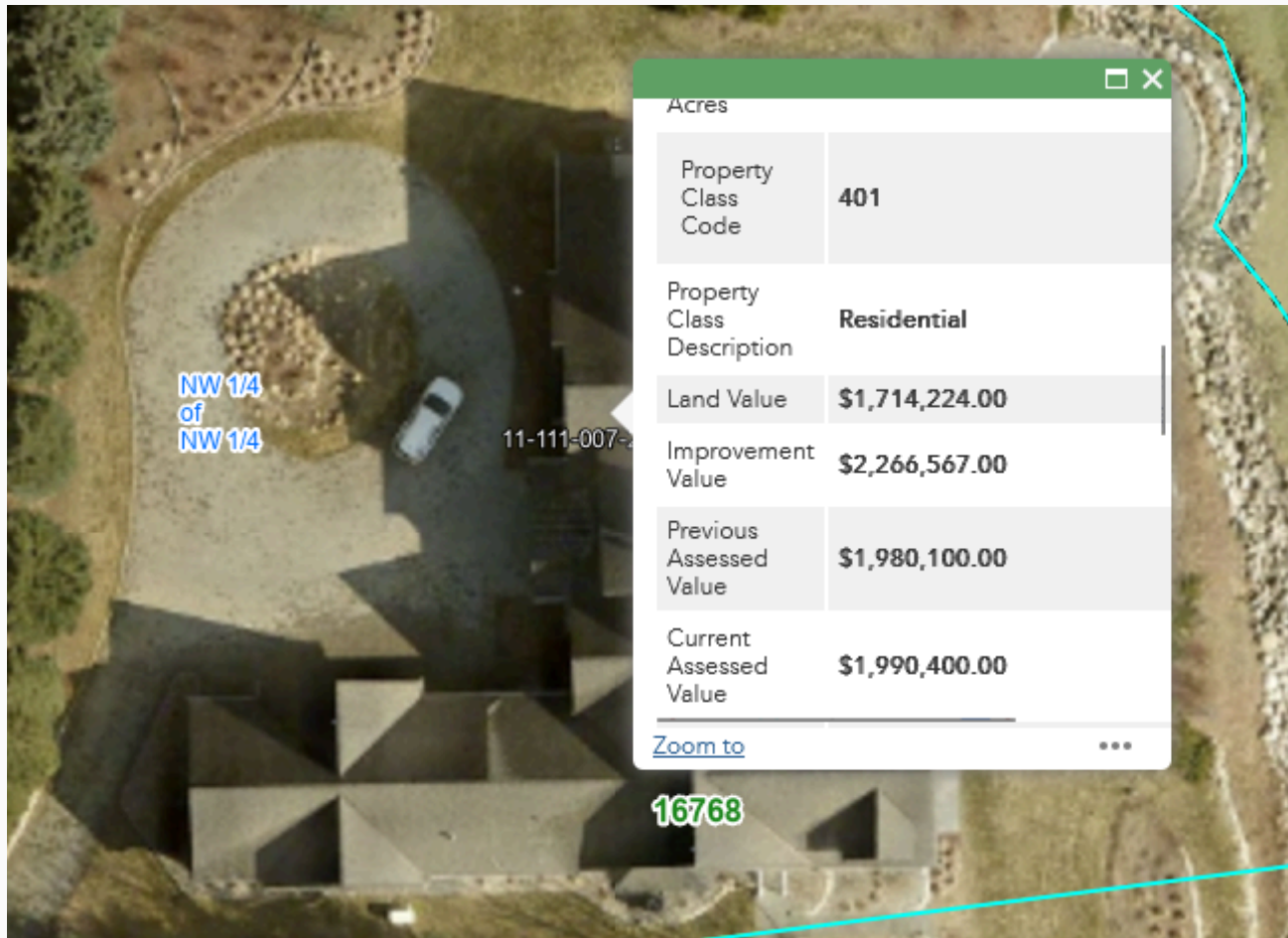
Application: Traverse City Properties

After checking the "I agree" box and clicking "OK", you click the dropdown to change the search type to "Parcel by ID", input the APN, and hit search.



Application: Traverse City Properties

This zooms you to the parcel shape *and* pulls up a ton of info, which includes the school district and current assessed/taxable home value!



Application: Traverse City Properties

Taking a step back, what is the workflow to go *from* an address to the displayed info on the parcel viewer map app?

1. Find the address' APN using the Property Info Site

- Plug in the street number
- Type the street address and select the autocomplete option
- Hit the search button
 - Wait for the page to load, then copy the APN

2. Find the property info for that APN on the Map Viewer

- Check the "I agree" box and click OK button
- Click the search dropdown and select "Parcel by ID"
- Enter the parcel's APN and click search button
- Copy the info that pops up in the window

Launch Automated Session

First, we need to **initiate an automated browser session** with

```
selenider_session().2
```

```
session ← selenider_session(  
  browser = "chrome", # browser to drive  
  session = "chromote", # set backend to chromote  
  timeout = 10 # set timeout to 10 sec  
)
```

We've now created a **local session**

- Accessed anywhere in a given script/RMarkdown file
- Closes automatically when the script/file finishes
- If started within a function, closes when the function is done running

² The full list of **selenider** functions can be found [here](#)

View the Session

If you want to **view the browser session**, we need to add the argument

```
options = chromote_options(headless = F)
```

Which will open the session in your Chrome browser so you can watch it in action!

```
session ← selenider_session(  
  browser = "chrome", # browser to drive  
  options = chromote_options(headless = F), # view the session  
  "chromote", # set backend to chromote  
  timeout = 10 # set timeout before error to 10 sec  
)
```

1. Navigate to Property Info Site

Let's start by navigating to the [Grand Traverse County Parcel Info Website](https://maps.grandtraverse.org/propertysearch.asp)

- `open_url("URL")`

```
open_url("https://maps.grandtraverse.org/propertysearch.asp")  
Sys.sleep(5)
```

1. Navigate to Property Info Site

Check that we ended up at the desired url:

```
current_url()
```

```
## [1] "https://maps.grandtraverse.org/propertysearch.asp"
```

Global Functions

Other actions that apply globally to the entire page:

Function	Task
<code>open_url()</code>	navigate to the indicated url
<code>current_url()</code>	get the current url
<code>get_page_source()</code>	get the page's HTML
<code>back()</code> , <code>forward()</code>	navigate forward or back
<code>reload()</code> , <code>refresh()</code>	reload the current page
<code>take_screenshot()</code>	take screenshot of current image
<code>execute_js_fn()</code> , <code>execute_js_expr()</code>	execute a JavaScript function

Selection Functions

Some additional selection functions:

Function	Task
<code>s()</code> , <code>ss()</code>	select HTML elements (without specifying the session)
<code>find_element()</code>	find a single HTML child element (yields a <code>selenium-element</code> object)
<code>find_elements()</code>	find multiple HTML child elements (yields a <code>selenium-elements</code> object)
<code>elem_filter()</code> , <code>elem_find()</code>	extract a subset of HTML elements

Selection Functions

Hold up James, you're telling me that there are **six** different functions??
Which should I be using?

Thoughts:

- `s()` is equivalent to `session %>% find_element()`
- `ss()` is equivalent to `session %>% find_elements()`
- `s()/find_element()` and `ss()/find_elements()` let you search by
 - CSS Selector (`css`)
 - XPath (`xpath`)
 - element ID (`id`), class (`class_name`), name (`name`)
- `elem_find()` and `elem_filter()` let us search by **conditions**
 - i.e. has certain text, has an attribute with a certain value, is visible, has a certain number of elements
 - We'll see helper functions that will help us out with this in a bit

Element Properties

And functions for obtaining element properties

Function	Task
<code>elem_attr()</code> , <code>elem_attrs()</code> , <code>elem_value()</code>	Get attributes of an element
<code>elem_css_property()</code>	Get a CSS property of an element
<code>elem_name()</code>	Get the tag name of an element
<code>elem_size()</code> , <code>length(seleniumider- elements)</code>	get the number of elements in a collection
<code>elem_text()</code>	Get the text inside an element
<code>elem_equal()</code>	Test if two elements are equal

Actions

Function	Task
<code>elem_click()</code> , <code>elem_double_click()</code> , <code>elem_right_click()</code>	Click on an element
<code>elem_hover()</code> , <code>elem_focus</code>	Hover over an element
<code>elem_scroll_to()</code>	Scroll to an element
<code>elem_select()</code>	Select an HTML element
<code>elem_set_value()</code> , <code>elem_send_keys()</code> , <code>elem_clear_value()</code>	Set the value of an input
<code>elem_submit()</code>	Submit an element
<code>keys</code>	list of special keys

1. Navigate the Property Info Site

Now let's think about the steps and functions need to **enter the address information** and then click hi-medgrn[Search] to try and track down the APN.

Involved Steps:

- **Find** the Address Number field (`find_element()`)
- **Type** the address number into the field (`elem_send_keys()`)
- **Find** the Street Name field (`find_element()`)
- **Type** the street name into the field (`elem_send_keys()`)
- **Click** the "Search" button (`find_element(), elem_click()`)

Fill the Address Number Field

Parcel Search

Parcel

Enter all or part of a parcel number, in the form of XX-XXX-XXX-XX.

Owner

Enter all or part of owner's name.

input#PROPADDNUM_id 896 × 44

```
session %>%  
  find_element(id = "PROPADDNUM_id") %>%  
  elem_send_keys("16768")
```

Fill the Street Name Field

Parcel Search

Parcel

Enter all or part of a parcel number, in the form of XX-XXX-XXX-XX.

Owner

Enter all or part of owner's name.

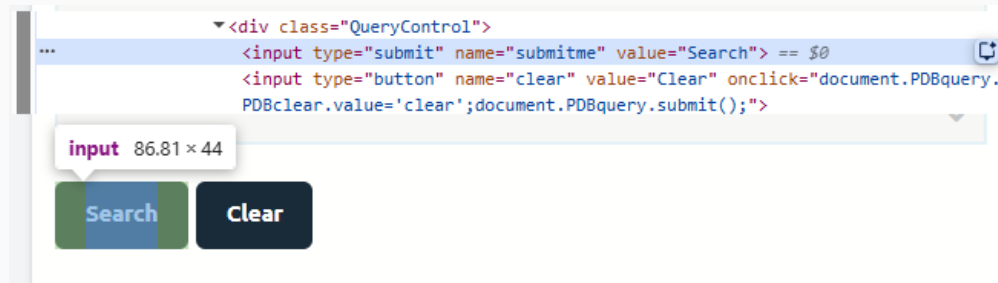
Address Number

Enter the address number of the physical address of the property you are searching for (example: 5480). To enter a list of Address numbers separate them by commas (example: 201,205,350). To enter a range of addresses separate them by the word 'to' (example: 201 to 301).

 **select#streetname_id** 896 × 44

```
session %>%  
  find_element(id = "streetname_id") %>%  
  elem_send_keys("Wrightwood Terrace Drive")
```

Click the Search Button



```
session %>%
  find_element(name = "submitme") %>%
  elem_click()
Sys.sleep(5)
```

Grab the APN

Parcel Search

1 item found.

You searched for items with Address Number = 16768, Street Name = WRIGHTWOOD TERRACE DR.

Parcel ▲	Owner ▲	Address ▲	City ▲
a.PDBlistlink	72.05 × 14		
11-111-007-25	SNIDER MANAGEMENT TRUST	16768 WRIGHTWOOD TERRACE DR	TRAVERSE CITY

```
apn ← session %>%  
  find_element("a.PDBlistlink") %>%  
  elem_text()  
apn
```

```
## [1] "11-111-007-25"
```

2. Navigate Map Viewer

Now we'll **click the "I agree to the above terms and conditions" checkbox** and then **click OK** to launch the app.

Involved Steps:

- **Select** the checkbox element by its CSS selector (`find_element()`)
- **Scroll to** the box (`elem_scroll_to()`)
- **Click** the box (`elem_click()`)
- Repeat to select/scroll to/click the OK button

2. Navigate the Map Viewer

Navigate to the main [Tax Mapping Viewer page](#) and launch the viewer

```
open_url("https://grand-  
traverse.maps.arcgis.com/apps/webappviewer/index.html?  
id=1f27e1d5c8bc4d8ea91e000305a8b6eb/")  
Sys.sleep(5)
```

2. Navigate the Map Viewer

Check that we ended up at the desired url:

```
current_url()
```

```
## [1] "https://grand-  
traverse.maps.arcgis.com/apps/webappviewer/index.html?  
id=1f27e1d5c8bc4d8ea91e000305a8b6eb/"
```

Click the "I Agree" Check Box

```
session %>%
```

```
  find_element("#jimu_dijit_CheckBox_0")  
  %>%  
    elem_text()
```

```
## [1] "I agree to the above terms and  
conditions"
```

```
session %>%  
  find_element("div.checkbox") %>%  
  elem_scroll_to() %>% # move the  
    cursor to the button  
  elem_click()
```



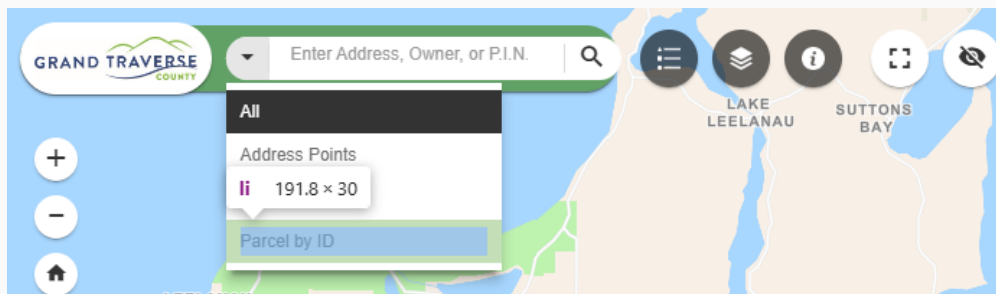
Click the "OK" Button

Alternative approach: the footer contains both the check box and the OK button

```
session %>%  
  find_element("div.footer") %>%  
  find_element("button") %>%  
  elem_scroll_to() %>% # move the  
cursor to the button  
  elem_click()  
Sys.sleep(5)
```



Choose the "Parcel by ID" Search



```
# Show the search options
```

```
session %>%
```

```
  find_element("#esri_dijit_Search_0_menu_button > span.searchIcon.esri-  
icon-down-arrow") %>%
```

```
  elem_click()
```

```
# select the fifth list element
```

```
session %>%
```

```
  find_element("#esri_dijit_Search_0 > div > div.searchMenu.sourcesMenu  
> ul > li:nth-child(5)") %>%
```

```
  elem_click()
```

Type in the APN



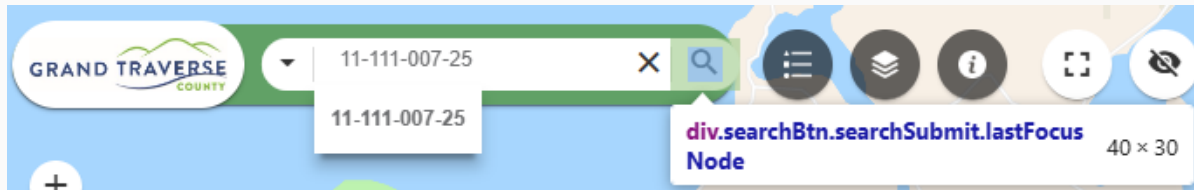
Show the search options

```
session %>%
```

```
  find_element("#esri_dijit_Search_0_input") %>%
```

```
  elem_send_keys(apn)
```

Press Search Button



Show the search options

```
session %>%
```

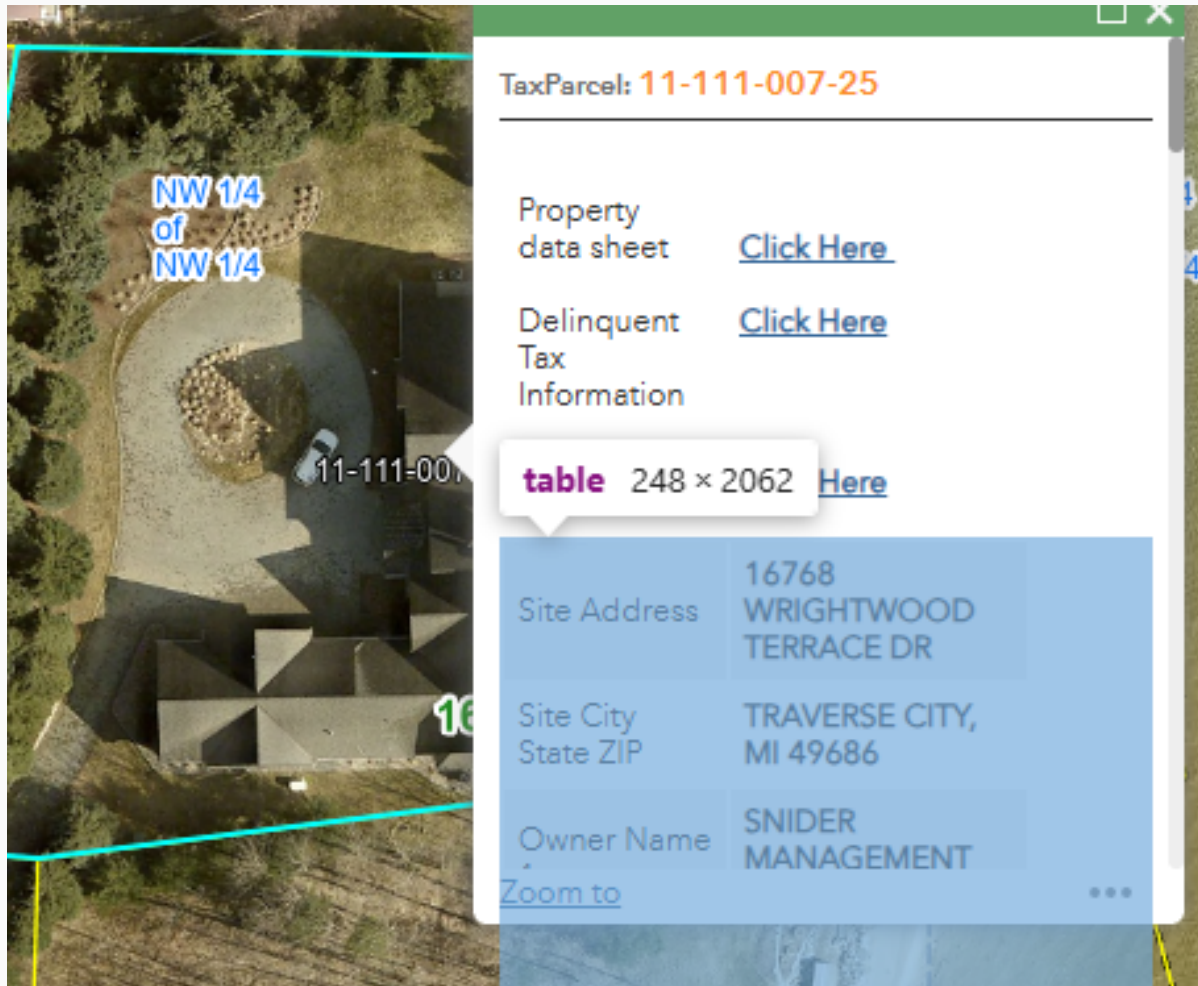
```
  find_element("div.searchBtn.searchSubmit.lastFocusNode") %>%
```

```
  elem_click()
```

```
Sys.sleep(5)
```

Get the Resulting Info Table

Here the info window contains several HTML Tables:



The screenshot shows an aerial map with a cyan rectangle highlighting a property. An info window is open over the property, displaying tax parcel information and a table of property details.

TaxParcel: 11-111-007-25

Property data sheet [Click Here](#)

Delinquent Tax Information [Click Here](#)

table 248 × 2062 [Here](#)

Site Address	16768 WRIGHTWOOD TERRACE DR
Site City State ZIP	TRAVERSE CITY, MI 49686
Owner Name	SNIDER MANAGEMENT

[Zoom to](#) ...

Get the Resulting Info Table

Here the info window contains several HTML Tables:

```
session %>%  
  find_elements("table")
```

```
## { selenider_elements (4) }  
## [1] <table><tbody><tr><td style="padding:5px;"><span style="font-size:14px;">Prop ...  
## [2] <table><tbody><tr style="background-color:rgb(242, 242, 242);"><td style="padding:5px;">Prop ...  
## [3] <table><tbody><tr><td style="padding:5px;"><span style="font-size:14px;">Prop ...  
## [4] <table class="dijit dijitMenu dijitMenuPassive dijitReset dijitMenuTable" rol ...
```

Get the Resulting Info Table

We can also use **condition functions** to help with selection.

Function	Task
<code>has_text(), has_exact_text()</code>	has text that partially or fully matches
<code>has_attr(), attr_contains(), has_value()</code>	does the attribute match a value
<code>has_css_property()</code>	does an element's CSS property match a value (class, id, title, etc.)
<code>has_name()</code>	does an element have a tag name
<code>has_length(), has_size(), has_at_least()</code>	does a collection have a certain number of elements
<code>is_present(), is_in_dom(), is_absent()</code>	does an element exist
<code>is_visible(), is_displayed(), is_hidden(), is_invisible()</code>	is an element visible

Get the Resulting Info Table

We can also use **condition functions** to help with selection.

- `find_elements()` to yield a `selenium_element`
- `elem_find()` to extract a specific subset of HTML elements
 - `has_text()` to grab the element with the text "Search"
- `elem_click()` to click the selected button.

```
session %>% find_elements("table") %>%
  elem_find(has_text("Site Address")) %>%
  elem_text()
```

```
## [1] "Site Address16768 WRIGHTWOOD TERRACE DR Site City State ZIP TRAVERSE CITY, MI 49686 Owner Name 1 SNIDER MANAGEMENT TRUST Owner Name 2 MICHAEL & DANA SNIDER TTEES Mailing Address 16768 WRIGHTWOOD TERRACE DR Mailing City TRAVERSE CITY Mailing State MI Mailing Zip 549686 Mailing Zip 4 Tax District Description Peninsula Township Tax District Code 11 School District Description TRAVERSE CITY SCHOOL DIST. School District Code 28010 Assessor Acres 1.10 ac Property Class Code 401 Property Class Description Residential Land Value $1,864,219.00 Improvement
```

Get the Resulting Info Table

selenium doesn't have built-in methods for reading HTML tables as data frames like we could in **rvest**...

... but we've already queried the website and had it populate the HTML with the info we need, so let's just pull the current HTML and use **rvest** methods as before!

```
get_page_source() %>%
  rvest::html_elements("table")

## {xml_nodeset (4)}
## [1] <table><tbody>\n<tr>\n<td style="padding:5px;">\n<span style="font-
size:1 ...
## [2] <table><tbody>\n<tr style="background-color:rgb(242, 242,
242);">\n<td st ...
## [3] <table><tbody><tr><td style="padding:5px;">\n<span style="font-
size:14px; ...
## [4] <table class="dijit dijitMenu dijitMenuPassive dijitReset
dijitMenuTable" ...
```

Get the Resulting Info Table

We can now grab the table that we want (2nd one in the nodeset). We can do this by

Full Selector

```
get_page_source() %>%  
  
rvest::html_element("div.mainSecti  
> div:nth-child(3) > figure:nth-  
child(2) > table") %>%  
  rvest::html_table()
```

```
## # A tibble: 28 × 3  
##       X1                X2  
X3  
##       <chr>  
<chr>  
<int>  
## 1 Site Address
```

Formatting all tables at once

Pass the full nodeset through
`html_table()` and use indexing

```
tabs ← get_page_source() %>%  
  rvest::html_elements("table")  
%>%  
  html_table()  
tabs[[2]]
```

```
## # A tibble: 28 × 3  
##       X1                X2  
X3  
##       <chr>
```

Get the Resulting Info Table

```
dplyr::select(tabs[[2]], X1, X2) %>%  
  dplyr::filter(str_detect(X1, "Mailing|School District|Value"))
```

```
## # A tibble: 13 × 2  
##       X1                                X2  
##   <chr>                             <chr>  
## 1 Mailing Address                    "16768 WRIGHTWOOD TERRACE DR"  
## 2 Mailing City                      "TRAVERSE CITY"  
## 3 Mailing State                     "MI"  
## 4 Mailing Zip 5                     "49686"  
## 5 Mailing Zip 4                     ""  
## 6 School District Description       "TRAVERSE CITY SCHOOL DIST."  
## 7 School District Code              "28010"  
## 8 Land Value                        "$1,864,219.00"  
## 9 Improvement Value                 "$3,542,251.00"  
## 10 Previous Assessed Value           "$1,990,400.00"  
## 11 Current Assessed Value            "$2,703,200.00"  
## 12 Previous Taxable Value            "$1,419,976.00"  
## 13 Current Taxable Value             "$1,458,315.00"
```

EX 2: Boston Marathon

Now that we've learned how to use dynamic scraping methods with **selenium**, let's go back to our initial motivation: scraping **2026 Boston Marathon results**

130TH BOSTON MARATHON[™]
BANK OF AMERICA

Results | Leaderboard | Tracking | My Runners

Results

▼ Search

Group: Runner ▼

Sub Group: Runner ▼

Last Name:

First Name:

Bib Number:

Search

Reset

► Advanced Search

Result Lists

Group: Runner ▼

Sub Group: Runner ▼

Gender: Men ▼

Age Group: All ▼

Results/Page: 25 ▼

Show Results

Force Close the Session

Before beginning, I find it helpful to **force quit the old session** and **start a new one**. This is especially helpful if it's been a while since the last actions and the driver's "stalled out" and become inoperable.

```
# Force the existing session to close
```

```
chromote::set_default_chromote_object(chromote::Chromote$new())
```

```
# start a new session
```

```
session ← selenider_session(
```

```
  options = chromote_options(headless = F), # headless = F to view the session
```

```
  "chromote", # set backend to chromote
```

```
  timeout = 10 # set timeout to 10 sec
```

```
)
```

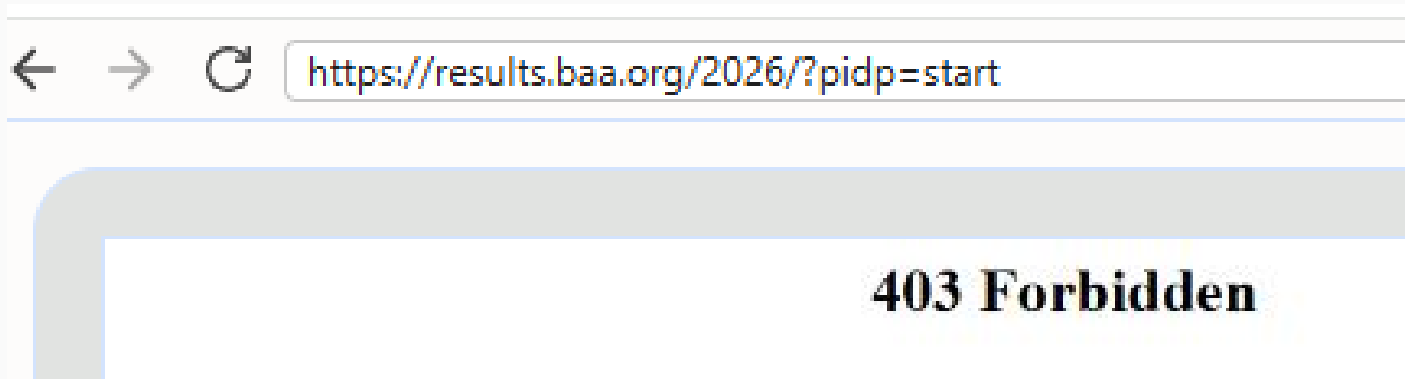
```
Sys.sleep(5)
```

Back to Boston Marathon

First, navigate to the [2026 Boston Marathon results search page](https://results.baa.org/2026/?pidp=start)

```
open_url("https://results.baa.org/2026/?pidp=start")
```

Dangit, site protections *again*.



Changing User Agent in selenider

As before, we can get around this by **changing the user agent**.

This time, it's slightly more complicated, as we need to modify settings of the chromote driver

- Adding a `$` after `session$driver` shows the immense amount of *stuff* we can look at/change
- Here we want to modify the `setUserAgentOverride()` buried in the `Network`
 - Sole argument `userAgent` is the user agent we want to use (same as last time)

```
session$driver$Network$setUserAgentOverride(  
  userAgent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64)  
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/93.0.4577.82  
Safari/537.36"  
)
```

(Back to) Back to Boston Marathon

Now try to navigate to the [2026 Boston Marathon results search page](https://results.baa.org/2026/?pidp=start)

```
open_url("https://results.baa.org/2026/?pidp=start")
Sys.sleep(5)
```

130TH BOSTON MARATHON[®]
BANK OF AMERICA

Results Leaderboard Tracking My Runners

Results

▼ Search

Group Runner ▼

Sub Group Runner ▼

Last Name

First Name

Bib Number

Search

Reset

► Advanced Search

Result Lists

Group Runner ▼

Sub Group Runner ▼

Gender Men ▼

Age Group All ▼

Results/Page 25 ▼

Show Results

Back to Boston Marathon

Let's suppose we want to see results for **every participant in Des Linden's age group (Women 40-44)**.

Workflow:

- Start a **selenider** session (done)
- Navigate to the **2026 Boston Marathon results search page** (done)
- Select the "Result Lists" form
- Use the "Gender" select element to select the "Women" option
- Use the "Age Group" element to select the "40-44" option
- Use the "Results/Page" dropdown to select 50

Select the Form

The first step before filling out the form is to identify the form that we want.

We *could* navigate an incredibly convoluted, highly-nested HTML structure, or we could

- Retrieve the source code and use **rvest** methods to identify a way to select the form we want

```
get_page_source() %>%  
  html_elements("form")
```

```
## {xml_nodeset (3)}  
## [1] <form class="form-horizontal" name="form_search_simple"  
id="form_search_s ...  
## [2] <form class="form-horizontal" name="form_search_advanced"  
id="form_search ...  
## [3] <form class="form-horizontal" name="form_lists_default"  
id="form_lists_de ...
```

Select the Form

Now we can use the `name` argument in `find_element()` to grab the "Results List" form.

```
find_element(session, name = "form_lists_default")

## { selenider_element }
## <form class="form-horizontal" name="form_lists_default"
## id="form_lists_default ...
##   \n<fieldset aria-label="fs-event" class="fs-event">\n<input
## type="hidden" n ...
## </form>
```

Or the `id` argument (conveniently both are the same here)

```
find_element(session, id = "form_lists_default")

## { selenider_element }
## <form class="form-horizontal" name="form_lists_default"
## id="form_lists_default ...
```

Form Fields

rvest's `html_form()` makes it easy to see the form's fields

```
get_page_source() %>%  
  
html_elements("form#form_lists_default")  
%>%  
  html_form()  
  
## [[1]]  
## <form> 'form_lists_default' (POST NA?  
pid=list&pidp=start)  
##   <field> (hidden) lang: EN_CAP  
##   <field> (hidden) startpage:  
start_responsive  
##   <field> (hidden) startpage_type: lists  
##   <field> (select) event_main_group:  
runner  
##   <field> (select) event: R  
##   <field> (select) search[sex]:  
##   <field> (select) search[age_class]: %  
##   <field> (select) num_results: 25  
##   <field> (button) submit:
```

Here we've got:

- 3 **hidden** fields
- 5 **select** fields that have **dropdown menus**
- 1 **button** to execute the search

Form Fields

Inspecting the HTML shows the same, albeit highly nested

```
▼<form class="form-horizontal" name="form_lists_default" id="form_lists_default" action="?pid=list&pidp=start" method="post"
  rt/form-data">
  ▼<fieldset aria-label="fs-event" class="fs-event">
    <input type="hidden" name="lang" id="default-lang" value="EN_CAP" class> == $0
    <input type="hidden" name="startpage" id="default-startpage" value="start_responsive" class>
    <input type="hidden" name="startpage_type" id="default-startpage_type" value="lists" class>
    ▼<div class="form-group type-select field-lists-event_main_group ">
      ::before
      <label class="col-sm-4 control-label" for="default-lists-event_main_group">Group</label>
      ▼<div class="col-sm-8">
        ▶<select placeholder class="form-control " name="event_main_group" id="default-lists-event_main_group">⋮</select>
        </div>
        ::after
      </div>
    ▼<div class="form-group type-select field-lists-event ">
      ::before
      <label class="col-sm-4 control-label" for="default-lists-event">Sub Group</label>
      ▼<div class="col-sm-8">
        ▶<select placeholder class="form-control " name="event" id="default-lists-event">⋮</select>
        </div>
        ::after
      </div>
    </fieldset>
```

Interacting with Form Fields

Within the form, we want to interact with the `select` elements. To do this, we will

1. Find the `select` element we're trying to fill in with `find_element()`
2. Find the select's `option` element that we want to select
3. Select it with `elem_select()`

Filling Form Fields

Let's start by setting the gender to Female.

I find the easiest way to identify the form options to be to

Right-click and "Inspect"

```
<div class="col-sm-8">
  <select placeholder class="form-control" name="sex"
    id="default-lists-sex">
    <option value="M">Men</option>
    <option value="W">Women</option>
    <option value="X">Non-Binary</option>
  </select>
</div>
```

Or to look at the options with
rvest:

```
get_page_source() %>%
  html_element("#default-lists-
sex") %>%
  html_elements("option")
```

```
## {xml_nodeset (3)}
```

```
[1] <option
value="M">Men</option>
```

Set Option Value

To set the option, use `elem_select()` on the desired option.

- Here we want a value of "W"

There are several different ways to do this:

1. XPATH `"//*/option[@value = \'VAL\']"`

- finds the `option` element with a `value` of `VAL`

```
find_element(session, name = "search[sex]") %>% # isolate the select field
```

```
find_element(xpath = "//*/option[@value = \'W\']") %>%  
elem_select()
```

Set Option Value

Next, let's set the age group to 40-44 (third option) by its position:

2. XPATH `"//*[@id = \"VALUE\"]/option[N]"`

- Finds the N^{th} option for the element with an `id` of `VALUE`
 - Here `N = 3`, `id = default-lists-age_class`

```
find_element(session, name = "search[age_class]") %>%  
  find_element(xpath = "//*[@id=\"default-lists-age_class\"]/option[3]")  
%>%  
  elem_select()
```

Set Option Value

Next, let's set the age group to 40-44 (third option)

3. CSS `"option:nth-child(N)"`

- Finds the `N`th option for the element with an `id` of `VALUE`

```
find_element(session, css = "#default-lists-age_class") %>% # isolate
the select field
  find_element(css = "option:nth-child(3)") %>%
  elem_select()
```

Back to Boston Marathon

If we're running headless, we can verify that our selection worked by viewing the age class search element

- Note which option now shows as "selected"

```
find_element(session, name =  
  "form_lists_default") %>%  
find_element(name =  
  "search[age_class]")  
  
## { selenider_element }  
## <select placeholder=""  
class="form-control"  
name="search[age_class]" id="defa  
...  
##   <option value=""  
selected="selected">All</option>  
<option value="1">18-39</ ...  
## </select>
```

```
<select placeholder class="form-control" name="search[age_class]"  
id="default-lists-age_class"> == $0  
  <option value="">All</option>  
  <option value="1" selected="selected">18-39</option>  
  <option value="2">40-44</option>  
  <option value="9">45-49</option>  
  <option value="3">50-54</option>  
  <option value="10">55-59</option>  
  <option value="4">60-64</option>  
  <option value="11">65-69</option>  
  <option value="8">70-74</option>  
  <option value="12">75-79</option>  
  <option value="13">80+</option>  
</select>
```

Set Option Value

Or combine the selectors together following `#ID > option:nth-child(N)`

```
find_element(session,  
  css = "#default-lists-age_class > option:nth-child(3)") %>%  
elem_select()
```

Set Option Value

Last, let's show 50 results per page

4. `elem_select(value = "VAL", reset_other = T)`

- Select the option with a `value` of `"VAL"`

```
find_element(session, "#default-num_results") %>% # grab the select  
  elem_select(value = "50")
```

Back to Boston Marathon

Finally, find and click the "Show Results" button!

```
find_element(session, name = "form_lists_default") %>%  
  find_element(name = "submit") %>%  
  elem_click()  
Sys.sleep(5)
```

```
▼ <fieldset class="fs-buttons">  
  ▼ <div class="form-group field-submit">  
    ::before  
    ▼ <div class="col-sm-offset-4 col-sm-8"> == $0  
      <button class="btn btn-primary selectorgadget_selected" type="submit"  
        name="submit" id="default-submit">Show Results</button>  
    </div>  
    ::after  
  </div>  
</fieldset>
```

Back to Boston Marathon

Our page has now been populated with the matching search results!

Retrieve the page's current HTML with `get_page_source()`, and then we can use **rvest** methods to retrieve the text we want.

```
Sys.sleep(5)
res_page1 ← get_page_source() %>%
  html_elements("div.row") %>%
  html_elements("li.list-group-item.row") %>%
  html_text2()
head(res_page1)
```

```
## [1] "Place Overall\nPlace Gender\nPlace
Division\nName\nTeam\nTeam\nBIB\nBIB\nHALF\nHALF\nFinish Net\nFinish
Net\nFinish Gun\nFinish Gun"
## [2] "418\n21\n1\nHall, Sara\nTeam\n-\nBIB\n111\nHALF\n01:11:03\nFinish
Net\n02:31:55\nFinish Gun\n02:31:55"
## [3] "706\n30\n2\nLinden, Desiree\nTeam\n-
\nBIB\n2018\nHALF\n01:16:47\nFinish Net\n02:35:49\nFinish Gun\n02:36:24"
## [4] "883\n36\n3\nPifer, Laura\nTeam\n-
```

Back to Boston Marathon

Challenge: format the page 1 results as a dataframe, splitting into all 9 columns

- Back to data wrangling/cleaning!
- *hint:* use `separate_wider_delim()`

Back to Boston Marathon

Challenge: format the page 1 results as a dataframe, splitting into all 9 columns

- Back to data wrangling/cleaning!
- *hint:* use `separate_wider_delim()`

Back to Boston Marathon

```
page1_df <- data.frame(string = res_page1[2:length(res_page1)]) %>%
  separate_wider_delim(cols = string, delim = "\n",
    names = c("place_overall", "place_gender",
      "place_division", "name", "team_dup", "team", "bib_dup", "bib",
      "half_dup", "half", "finish_net_dup", "finish_net", "finish_gun_dup",
      "finish_gun")) %>%
  dplyr::select(-ends_with("dup"))
```

```
head(page1_df)
```

```
## # A tibble: 6 × 9
```

```
##   place_overall place_gender place_division name      team  bib    half
finish_net
##   <chr>          <chr>          <chr>          <chr>    <chr> <chr> <chr>
<chr>
## 1 418           21            1            Hall, ... -    111  01:1...
02:31:55
## 2 706           30            2            Linden... -    2018  01:1...
02:35:49
## 3 883           36            3            Pifer,... -    168  01:1...
02:37:33
```

Scraping Dynamic/Interactive Websites

Methods for scraping dynamic or interactive websites opens up a huge range of possibilities.

- Iterative process of
 - Interact with the website and its elements
 - Extract element contents as with static sites
- Interactive workflows are generally more complex, so worth thinking about tradeoff between just brute forcing vs. coding up a scraping routine

Things will also get exponentially easier once we get through our programming lecture, which will allow you to automate repeated scraping tasks

Table of Contents

1. Prologue
2. Scraping Dynamic Websites